First Exercise: DCNN & Vision Transformer for Hand Gesture Recognition



Aristotle University of Thessaloniki

Digital Media – Computational Intelligence

Stelios Farsarakis – AEM: 159

DMCI201/AI204 – Deep Learning and Multimedia Information Analysis

Description of Algorithm, Model Types and Results

For this exercise, I decided to use these architectures: Deep Convolutional Neural Networks and Vision Transformer (ViT) to train and evaluate them to classify static hand-gesture images. The exercise was split into two parts:

a. The part that only works in Windows environment and
b. The part that only works in Linux Environment.

The main "AI204_DCNN_ViT_Farsarakis_Project1.ipynb" file can only be executed in linux environment because of its logic used. For faster training I decided to switch to GPU training instead of CPU. So, I used TensorFlow 2.21.0 with NVIDIA GPU acceleration.

I created a personal Jupyter local server for that reason to run my file. These steps were automatically run after writing "dmci201-jupyter":

1. source .venv/bin/activate
2. jupyter notebook --no-browser --ip=127.0.0.1 --port=8888

The server was necessary because we wanted the notebook to run with our GPU TensorFlow setup. The notebook itself does not choose the GPU environment automatically. The kernel/server must come from the environment where GPU TensorFlow is installed. So, after executing the steps above we use that environment and NVIDIA GPU can be detected.

The main idea of the project was to create a personal dataset of hand gestures: “open palm”, “fist” and “thumbs up”. For that reason, I used a file named “collect_custom_hand_gestures.py” that was run locally in my windows environment and uses a mechanism which captures X images (e.g. 200) of each hand gesture given as a parameter. A way to run this file is:

python collect_custom_hand_gestures.py --class-name thumbs_up --target 200 --output data/custom_hand_gestures

After running the program, camera opens and a green box is shown to which we place our hand and continue capturing images of a particular gesture. Images are saved inside the directory we give in –output (e.g. data/custom_hand_gestures).

Then I just printed the directories where the dataset is placed (either existing or custom). I decided to capture ~600 images per hand gesture and plot a histogram (just to show the class balance)

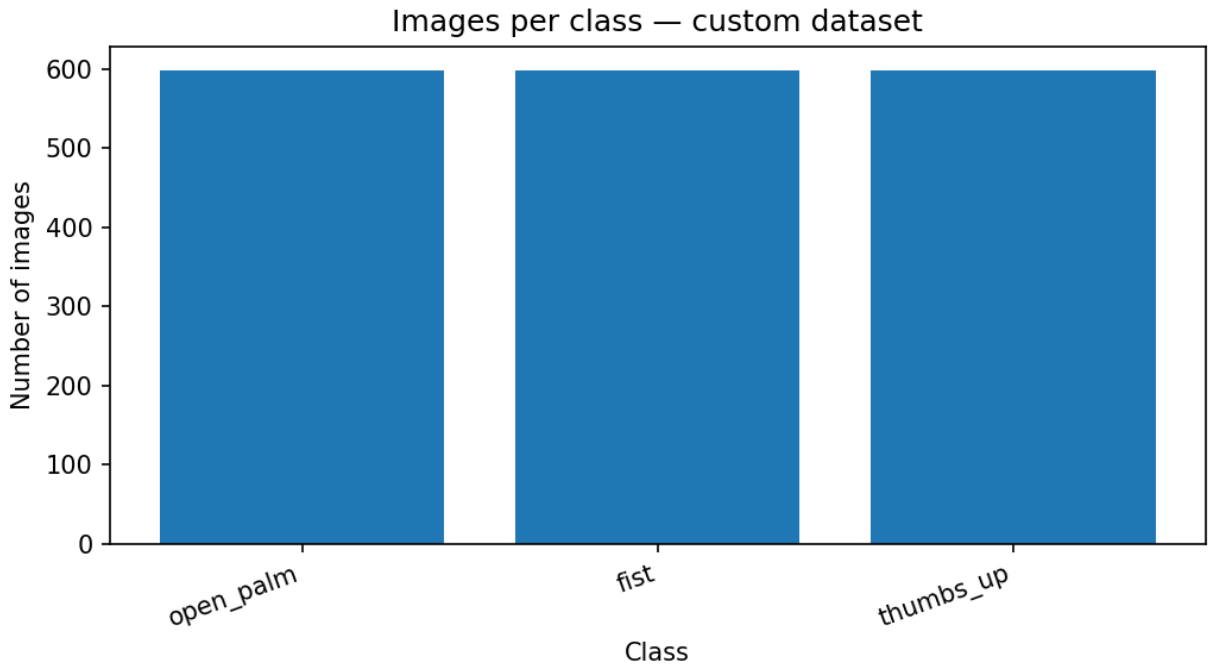


Figure 1: Showcase of the number of images per class

Then I decided to make a function just to show some random examples of images per class:



Figure 2: Grid of random example of images per class of my own dataset

We did a division of dataset into training, validation and test subsets using a stratified split. First, 15% of the images were kept as a test set. Then, the remaining 85% was split again so that approximately 15% of the original dataset was used for validation and 70% for training. Stratification was used to preserve the class distribution across all subsets.

Then, the image paths and their labels were converted into TensorFlow datasets using the tf.data API. Each image was read from disk, decoded as an RGB image, resized to the model input size, and normalized by scaling pixel values from [0, 255] to [0, 1]. The training dataset was shuffled at each epoch to avoid learning from a fixed image order, while the validation and test datasets were kept unshuffled. Finally, batching and prefetching were applied to improve training efficiency, especially when using GPU acceleration.

In the next section (8.), two different deep learning architectures are implemented for the same hand-gesture classification task: a custom DCNN and a custom Vision Transformer. Both models receive RGB images of size 128 × 128 × 3 and output three class probabilities using a softmax layer. Before entering the model, the images pass through a data augmentation pipeline that applies random horizontal flips, small rotations, zooming, and contrast changes. This improves generalization because the hand may appear in different positions, sizes, and lighting conditions during live webcam usage.

The DCNN consists of four convolutional blocks with 32, 64, 128, and 256 filters. Each block uses convolution, batch normalization, and ReLU activation, while the first three blocks also use max pooling to reduce the spatial dimensions of the feature maps. The convolutional layers learn local visual patterns such as edges, contours, fingers, palm shapes, and thumb positions. After the convolutional part, global average pooling converts the feature maps into a compact vector, followed by dropout, a dense layer, and a final softmax classification layer. The model summary shows that the image size is gradually reduced from 128 × 128 to 16 × 16 before classification, and the final output shape is (None, 3).

The Vision Transformer follows a different approach. Instead of processing the image mainly through convolutional blocks, it divides each 128 × 128 image into 16 × 16 patches, producing 64 patches in total. Each patch is projected into a 64-dimensional vector, and positional embeddings are added so that the model knows the spatial order of the patches. The patch sequence is then processed by a Transformer block with layer normalization, multi-head self-attention, residual connections, and an MLP. Through self-attention, the model can learn relationships between different regions of the hand image, such as the relative position of the palm, fingers, and thumb. Finally, global average pooling and a softmax layer are used to classify the gesture.

The notebook uses the variable MODEL_TYPE to select which architecture will be trained. If MODEL_TYPE = "dcnn", the DCNN is built; if MODEL_TYPE = "vit", the Vision Transformer is built. Both models are compiled with the Adam optimizer, sparse categorical crossentropy loss, and accuracy as the evaluation metric. This allows a fair comparison between the DCNN and the Vision Transformer because both architectures are trained and evaluated using the same dataset and preprocessing pipeline.

During training in section 9, three callbacks were used to make the process more reliable. ModelCheckpoint saves the best model according to validation accuracy, so the final saved model corresponds to the best validation performance and not necessarily to the last epoch. EarlyStopping monitors the validation loss and stops training if it does not improve for several epochs, restoring the best weights afterwards. This helps avoid

unnecessary training and reduces the risk of overfitting. ReduceLROnPlateau also monitors the validation loss and reduces the learning rate when the model stops improving, allowing training to continue with smaller and more careful weight updates. Finally, the model is trained with model.fit() using the training dataset and the validation dataset. The returned history object stores the loss and accuracy values for each epoch, which are later used to create the training and validation curves.

For the DCNN model, the training curves show that the model learns the task very successfully.



Figure 3: Training and validation losses in 30 epochs for DCNN model

In the loss curve, the training loss decreases quickly from approximately 0.95 to below 0.2. The validation loss is unstable during the first epochs and reaches high values at the beginning, but after around epoch 12 it decreases sharply and becomes very close to the training loss. By the final epochs, both training and validation loss are low and stable.

This is a positive result because it means that the model is not only memorizing the training set, but also generalizes well to the validation set.

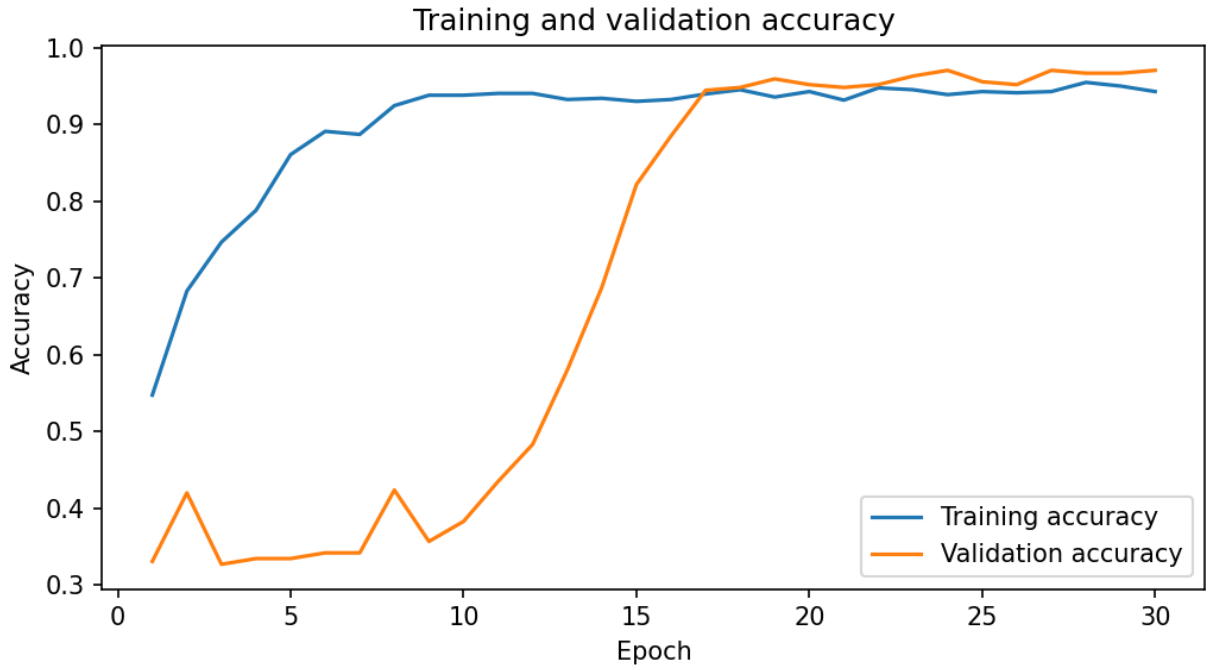


Figure 4: Training and validation accuracies in 30 epochs for DCNN model

In the accuracy curve, the training accuracy increases rapidly from around 55% to above 90%. The validation accuracy starts low, close to random behavior, but after several epochs it increases strongly and reaches approximately 95–97%. This indicates that the DCNN successfully learned the visual differences between the three hand gestures.

A useful observation is that the validation accuracy becomes slightly higher than the training accuracy in the final epochs. This is not necessarily a problem. During training, data augmentation and dropout are active, which makes the training task harder. During validation, augmentation and dropout are not applied in the same way, so the validation performance can sometimes appear slightly better.

Overall, the DCNN curves show strong learning, stable convergence, and no serious overfitting.

The Vision Transformer shows a different behavior. In the accuracy curve, both training and validation accuracy improve gradually, but they remain much lower than the DCNN. The training accuracy starts close to random guessing and increases slowly, reaching approximately 53–54% by the final epochs. The validation accuracy fluctuates but also improves and reaches around 50–51%.

Since this is a three-class classification problem, random guessing would be approximately: 1/3 = 33.333%

Therefore, the Vision Transformer performs better than random guessing, but it does not learn the task as effectively as the DCNN. The loss curve shows a gradual decrease in both training and validation loss. The training loss decreases from around 1.6 to below 1.0, while the validation loss also slowly decreases and remains close to the training loss. This means that the ViT is learning something useful, but its learning is slower and weaker compared to the DCNN.

The important point is that the Vision Transformer does not show strong overfitting. The training and validation curves remain close to each other. Instead, the main issue is underfitting: the model does not achieve high training accuracy, which means it has not fully learned the dataset.

This result is reasonable because Vision Transformers usually require larger datasets to perform well. In this project, the dataset is small and custom-made, so the DCNN has an advantage because convolutional networks are naturally well suited for image tasks and can learn useful local patterns from fewer images.

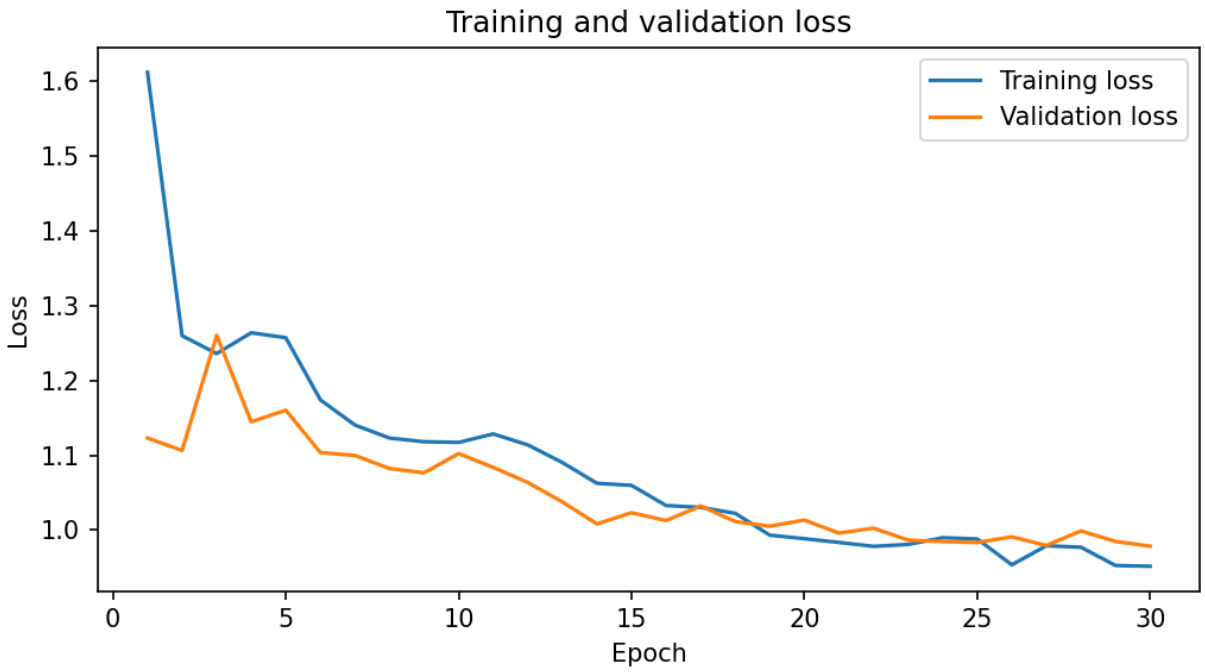


Figure 5: Training and validation losses in 30 epochs for ViT model

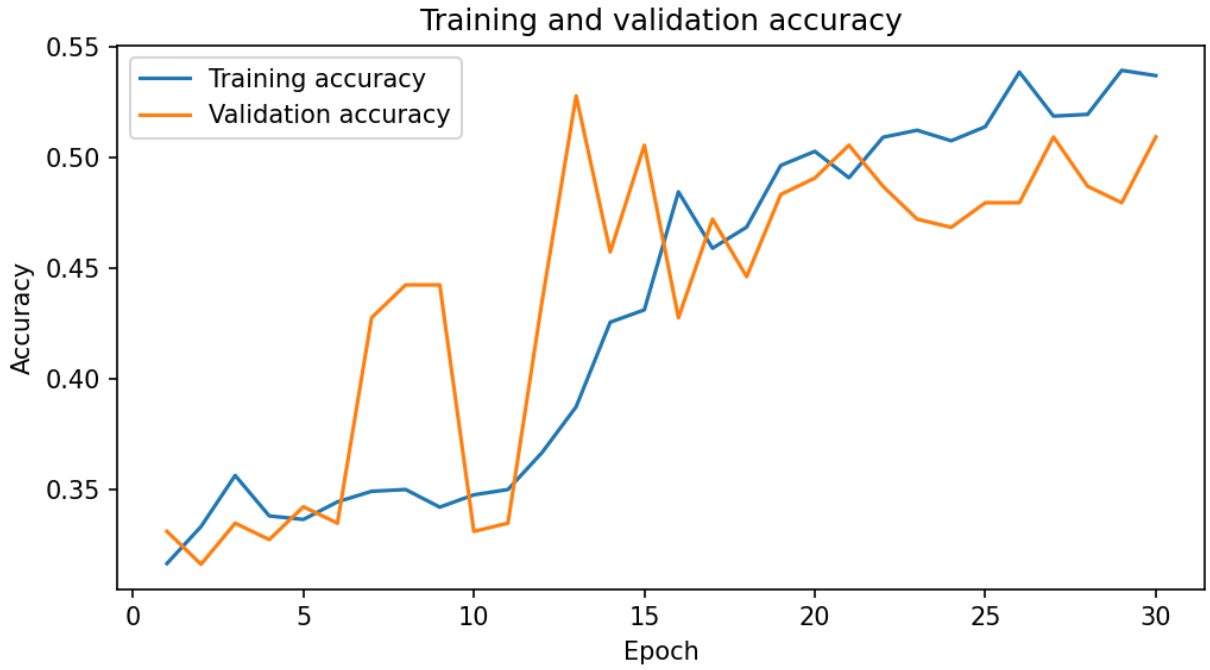


Figure 6: Training and validation accuracies in 30 epochs for ViT model

The comparison between the two models shows that the DCNN performs much better on this custom hand-gesture dataset.

The DCNN reaches high training and validation accuracy, while the Vision Transformer only reaches around 50% validation accuracy. This is expected because the dataset is relatively small. DCNN have a strong inductive bias for images: they are designed to learn local spatial patterns such as edges, contours, and object parts. This makes them very effective for small image datasets.

The Vision Transformer, on the other hand, treats the image as a sequence of patches and learns relationships between patches using self-attention. This is powerful, but it usually requires more data to train effectively from scratch. In this project, the custom ViT model was able to perform above random chance, but it did not reach the performance of the DCNN.

After training in section 11, the best saved model is loaded from disk. During training, ModelCheckpoint saved the model that achieved the best validation accuracy. This block reloads that best model before testing. This is important because the final epoch is not always the best-performing epoch. By loading the best saved model, the test evaluation is performed on the strongest version of the trained architecture. Then the model is evaluated

on the independent test set. The test set was not used during training or validation. Therefore, this evaluation gives a more realistic estimate of how well the model performs on unseen images.

For the **DCNN**, the test result was:

Test loss → 0.1529

Test accuracy → 0.9556

This means that the DCNN correctly classified approximately **95.56%** of the test images. The low test loss also shows that the model predictions are confident and close to the correct labels.

For the Vision Transformer, the test result was:

Test loss → 1.0381

Test accuracy → 0.5

This means that the ViT correctly classified **50%** of the test images. Since the task has three balanced classes, random guessing would be around **33.3%**. Therefore, the Vision Transformer performs better than random guessing, but much worse than the DCNN.

The next function (collect_predictions(model, ds) in section 12) collects the true labels, predicted labels, and predicted probabilities from the test set. For each batch, the model produces probabilities for the three classes. For example, one prediction could look like:

[0.10, 0.85, 0.05]. This means the model assigns:

10% probability to class 0

85% probability to class 1

5% probability to class 2

For DCNN:

We create a classification report:

	precision	recall	f1-score	support
open_palm	1.0000	0.9778	0.9888	90
fist	0.9158	0.9667	0.9405	90
thumbs_up	0.9540	0.9222	0.9379	90
accuracy			0.9556	270
macro avg	0.9566	0.9556	0.9557	270
weighted avg	0.9566	0.9556	0.9557	270

Figure 7: Classification report for DCNN

This is a very strong result. The class open_palm has:

Precision → 100%

Recall → 97.78%

F1-Score → 98.88%

This means that when the model predicted open_palm, it was always correct, and it found almost all real open_palm examples.

The class fist has:

Precision → 91.58%

Recall → 96.67%

F1-Score → 94.05%

This means the model detected most fist images correctly, but a few predictions of fist may have actually belonged to another class.

The class thumbs_up has:

Precision → 95.4%

Recall → 92.22%

F1-Score → 93.79%

This is also strong, although slightly weaker than open_palm.

The macro average F1-score is: 95.57%

And this confirms that the DCNN performs well across all three classes, not only on one class.

For Vision Transformer:

	precision	recall	f1-score	support
open_palm	0.4262	0.8667	0.5714	90
fist	0.8333	0.3333	0.4762	90
thumbs_up	0.5294	0.3000	0.3830	90
accuracy			0.5000	270
macro avg	0.5963	0.5000	0.4769	270
weighted avg	0.5963	0.5000	0.4769	270

Figure 8: Classification report for Vision Transformer (ViT)

For open_palm:

Precision → 42.62%

Recall → 86.67%

F1-Score → 57.14%

The recall is high, which means the model finds many real open_palm images. However, the precision is low, which means it also incorrectly predicts many other images as open_palm. So the model seems biased toward predicting open_palm.

For fist:

Precision → 83.33%

Recall → 33.33%

F1-Score → 47.62%

The precision is high, but recall is low. This means that when the model predicts fist, it is often correct, but it misses many real fist examples.

For thumbs_up:

Precision → 52.94%

Recall → 30%

F1-Score → 38.30%

This is the weakest class. The model struggles to correctly identify many thumbs_up images.

Model	Test accuracy	Macro F1-score	Interpretation
DCNN	95.56%	0.9557	Strong classification performance
ViT	50.00%	0.4769	Above random, but weak performance

To sum up, The final evaluation shows that the DCNN is clearly the better model for this dataset. It achieves high accuracy and high F1-scores for all three classes. The Vision Transformer performs above random chance but remains much weaker, mainly because it underfits the dataset. This result is reasonable because the dataset is relatively small and custom-made. CNNs are well suited for small image datasets because they learn local visual features efficiently, while Vision Transformers usually require larger datasets or pretraining to achieve strong performance. Therefore, the DCNN was selected as the most suitable model for the live hand-gesture detection system.

Confusion Matrices:

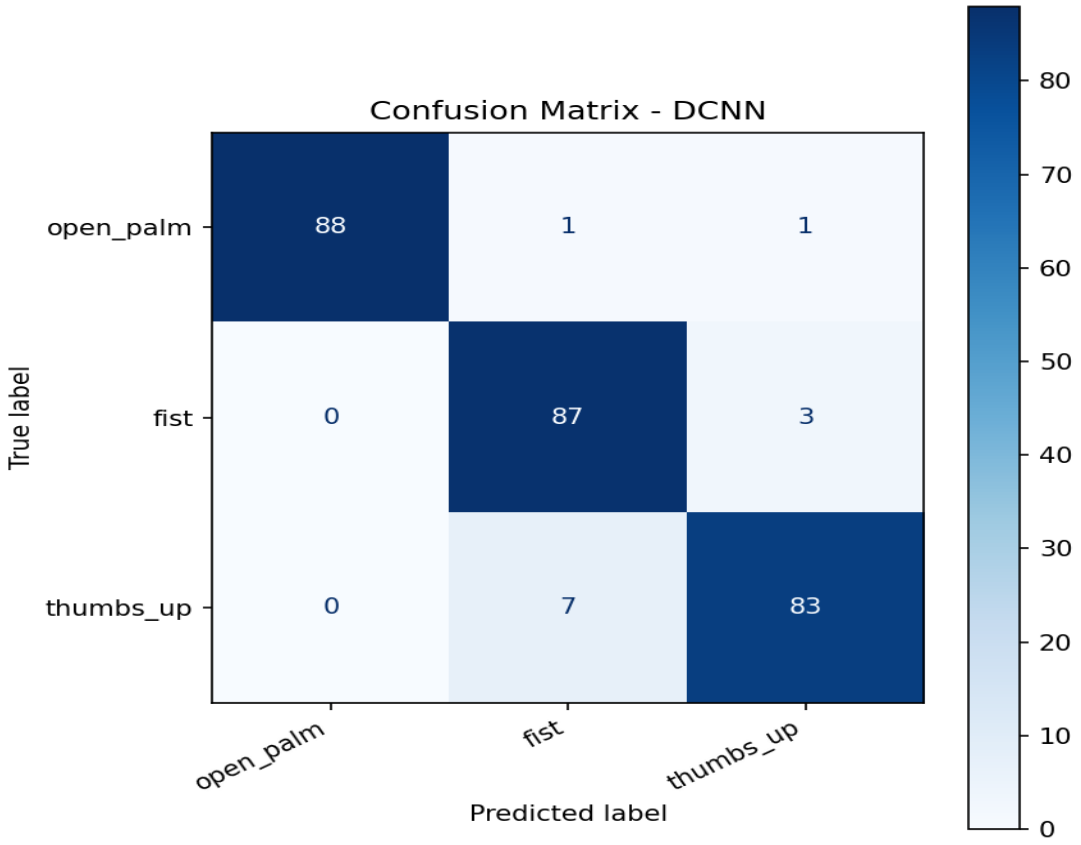


Figure 9. Confusion matrix of the DCNN model on the test set. The model correctly classifies most samples in all three gesture classes, with 88/90 correct predictions for open_palm, 87/90 for fist, and 83/90 for thumbs_up. Most errors occur between thumbs_up and fist, while open_palm is almost perfectly separated from the other classes.

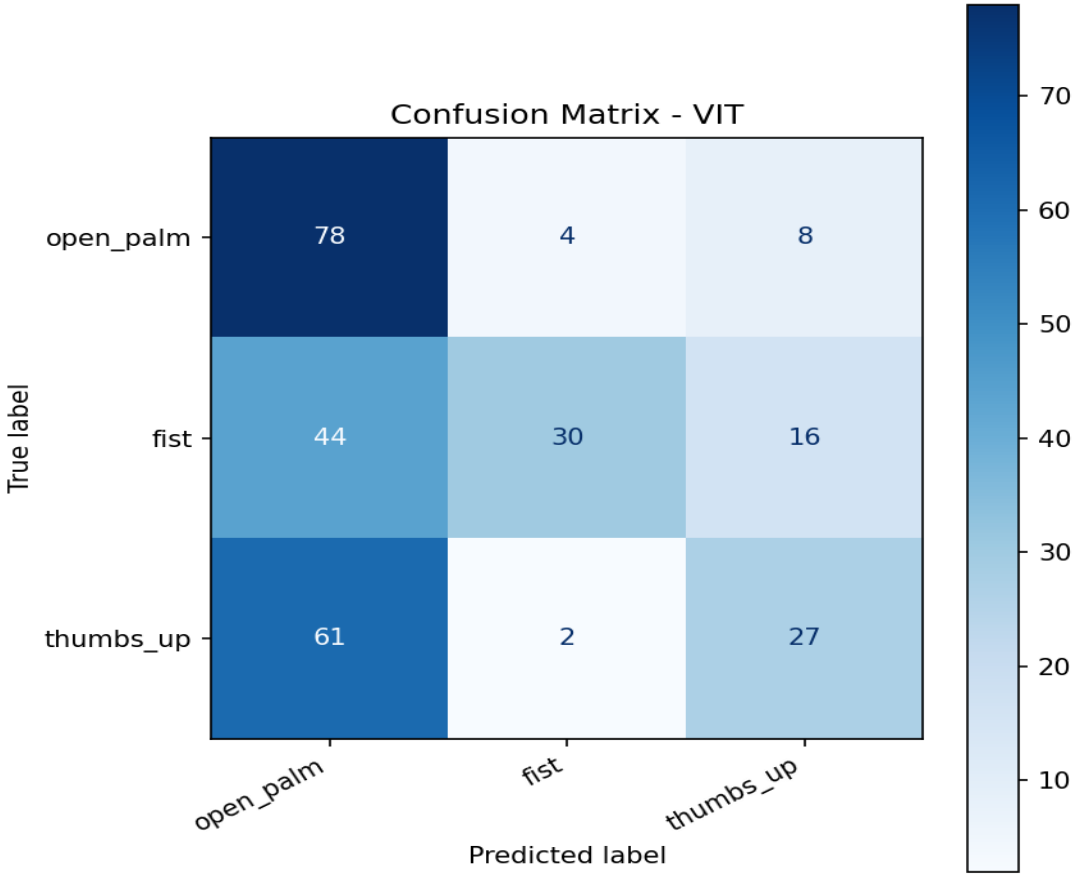


Figure 10. Confusion matrix of the Vision Transformer model on the test set. The ViT correctly identifies many open_palm samples, but it frequently misclassifies fist and thumbs_up images as open_palm. This shows that the ViT learned some useful patterns, but it did not separate the three gesture classes as effectively as the DCNN.

Section 12 is used to inspect the wrong predictions visually. This helps understand the type of mistakes each model makes. The function predict_single_image() loads one image, applies the same preprocessing as the training pipeline, and returns the predicted class. The image is read from disk, decoded as an RGB image, resized to the model input size, and normalized to the range [0, 1]. Then, the model predicts the class probabilities. The class with the highest probability is selected using np.argmax. Next, we find which test images were classified incorrectly.

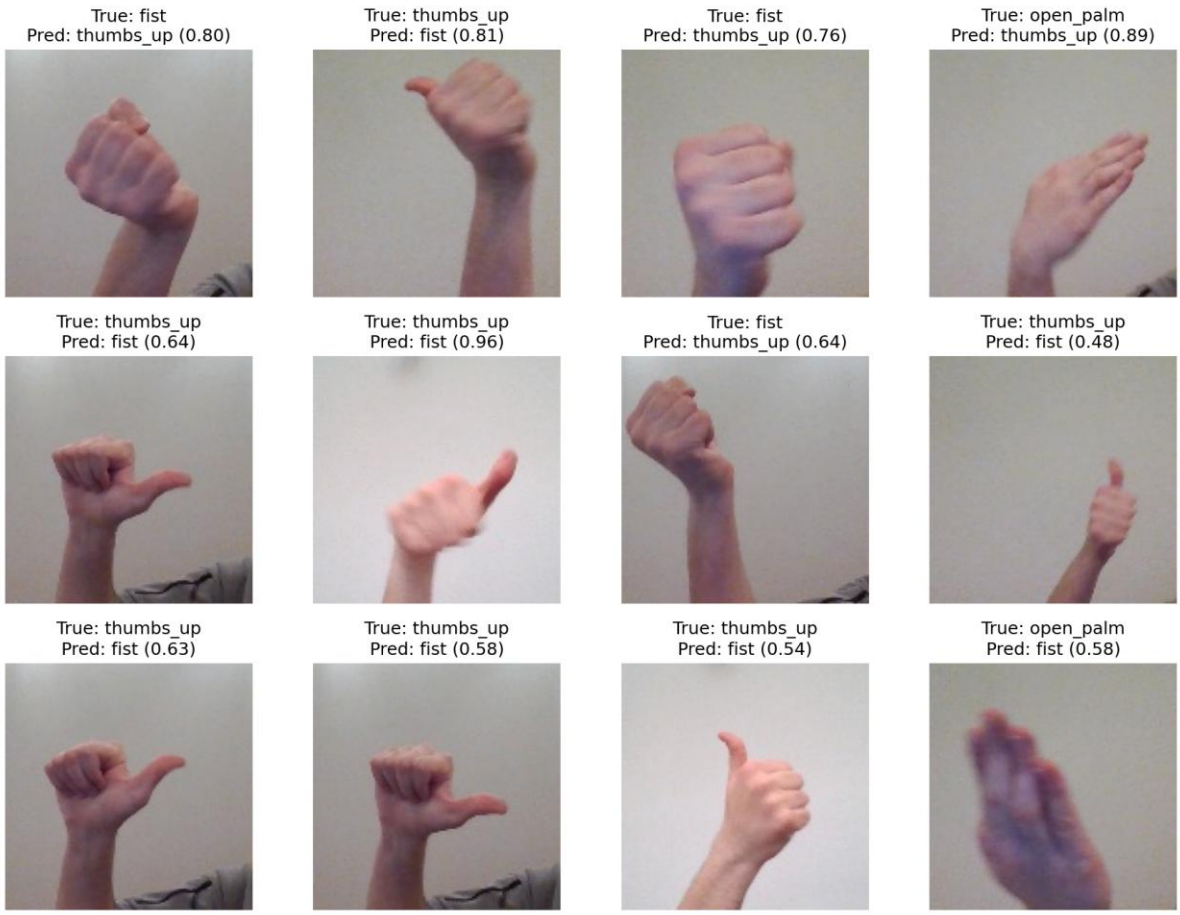


Figure 11. Examples of incorrect predictions made by the DCNN model. Most errors occur between the visually similar fist and thumbs_up classes, while open_palm is rarely misclassified. This shows that the DCNN performs well overall and mainly struggles with ambiguous hand poses.

For the DCNN, the number of incorrect predictions is small (12). The displayed wrong predictions show that most errors happen between fist and thumbs_up. This is reasonable because these two gestures can sometimes look visually similar, especially when the thumb is not very clearly visible or when the hand is rotated.

Some examples show that the DCNN predicts thumbs_up when the true class is fist, and others show the opposite. This agrees with the confusion matrix, where most DCNN mistakes were between these two classes.

The DCNN also makes a few mistakes with open_palm, but these are rare. In general, the incorrect prediction examples confirm that the DCNN learned the task well and only struggles with visually ambiguous cases.

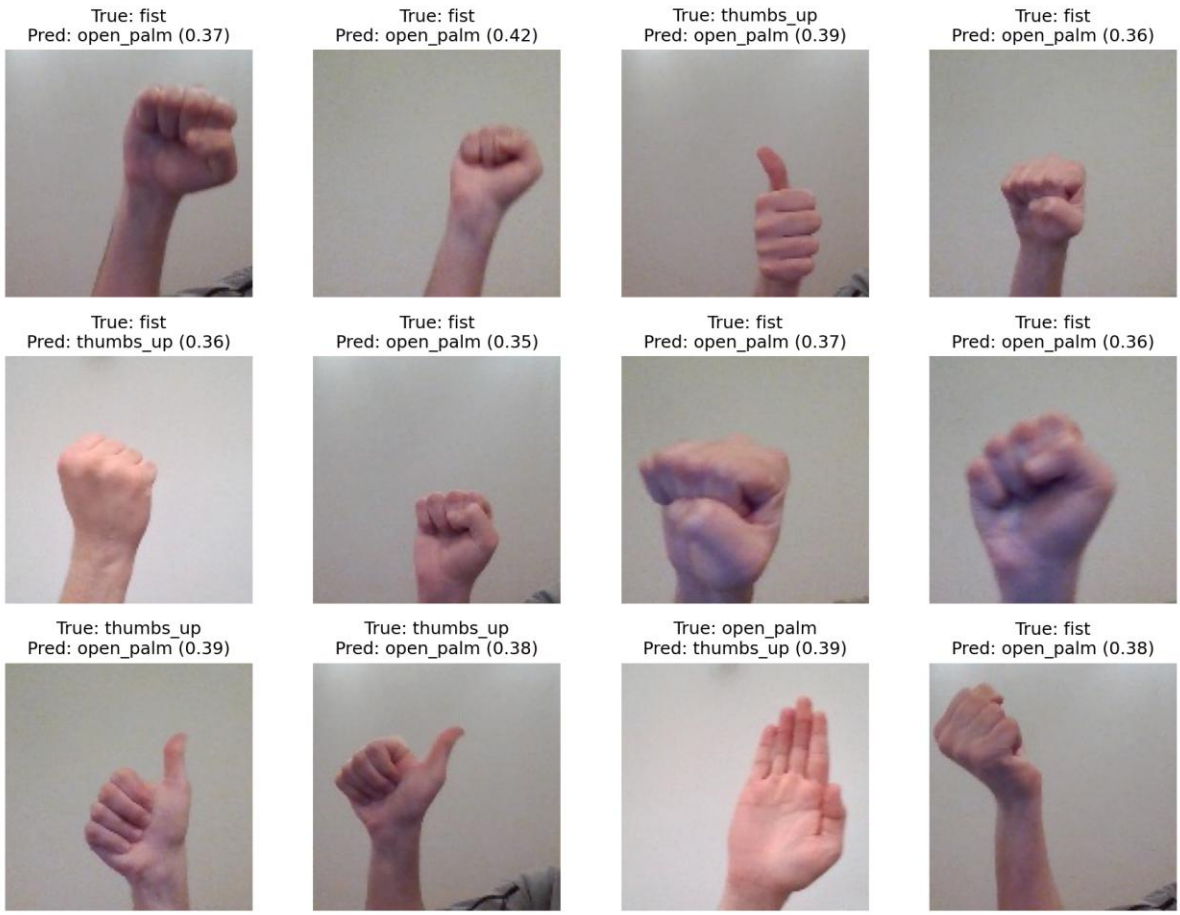


Figure 12. Examples of incorrect predictions made by the Vision Transformer model. The ViT frequently misclassifies fist and thumbs_up images as open_palm, showing that it does not separate the three gesture classes as effectively as the DCNN.

For the Vision Transformer, the incorrect prediction examples show a different pattern. Many images from fist and thumbs_up are predicted as open_palm. This agrees with the ViT confusion matrix, where the model shows a strong tendency to predict open_palm.

Also, many ViT wrong predictions have relatively low confidence values, such as around 0.35–0.42. This means that the model is not very certain about its decisions. In contrast, the DCNN often makes fewer mistakes, but some of its mistakes are made with higher confidence.

The ViT examples show that the model has learned some visual information, but it has not learned the gesture boundaries clearly enough. It struggles especially with distinguishing fist and thumbs_up, and it often assigns them to open_palm.

After training and evaluating the models, the final trained model was saved to disk using the Keras .save() method. This allows the model to be reused later without retraining. In addition, the class names were saved in a JSON file, so that the numerical output of the model can be mapped back to the correct gesture labels. This is necessary for the live detection script, where the model prediction must be displayed as a human-readable class name such as open_palm, fist, or thumbs_up.

In section 14, a helper function was also implemented to test the trained model on a single image. The image is loaded, resized, normalized, and passed through the model. The predicted class is displayed above the image, together with the confidence score. The function also prints the probability assigned to each gesture class. This provides a simple way to inspect individual predictions and verify that the trained model can be used outside the training loop.

Custom dataset collection script

In addition to the notebook, I implemented a separate Python script named collect_custom_hand_gestures.py for collecting my own hand-gesture dataset using a webcam. The script supports the three selected classes of the project: open_palm, fist, and thumbs_up. These are defined as the valid class names, so the user cannot accidentally collect images for an unsupported class.

The script is executed separately for each gesture class. For example:

python collect_custom_hand_gestures.py --class-name open_palm --target 200 --output data/custom_hand_gestures

python collect_custom_hand_gestures.py --class-name fist --target 200 --output data/custom_hand_gestures

python collect_custom_hand_gestures.py --class-name thumbs_up --target 200 --output data/custom_hand_gestures

For each class, the webcam opens and a rectangular region of interest is shown on the screen. The user places the hand inside this region, and the script saves only the selected region instead of the full webcam frame. Each captured image is resized to 128 × 128, which matches the input size used later by the DCNN and Vision Transformer models.

The script supports both automatic and manual image capture. Pressing SPACE starts or stops automatic capture, pressing c captures a single image manually, and pressing q exits the program. This made it possible to collect many images quickly while still controlling the quality of the dataset. The images are saved into class-specific folders, for example:

data/custom_hand_gestures/open_palm

data/custom_hand_gestures/fist

data/custom_hand_gestures/thumbs_up

The script also avoids overwriting existing images by checking the next available image index before saving new files. This allowed me to collect the dataset gradually and add more images later if needed.

This part of the implementation is important because it shows that the dataset was not only downloaded or reused, but also created and organized specifically for this project.

Live webcam detection script

I also implemented a second Python script named live_detection.py, which uses the trained model for real-time hand-gesture recognition through the webcam. This script loads a saved Keras model and the corresponding class names file, opens the webcam, extracts a region of interest, preprocesses it, and displays the predicted gesture together with its probability.

This script can be executed with a command such as:

python live_detection.py --model models/hand_gesture_dcnn_custom_final.keras --class-names models/hand_gesture_dcnn_custom_class_names.json

The live detection script uses the same preprocessing logic as the notebook. The webcam image is cropped to the selected region of interest, converted from OpenCV's BGR format to RGB, resized to the model input size, converted to float32, and normalized to the range [0, 1]. This is necessary because the input format during live inference must match the format used during training.

After preprocessing, the image is passed to the trained model, which outputs probabilities for the three gesture classes. The predicted class is selected using the highest probability. The script then displays the result on the webcam window, for example:

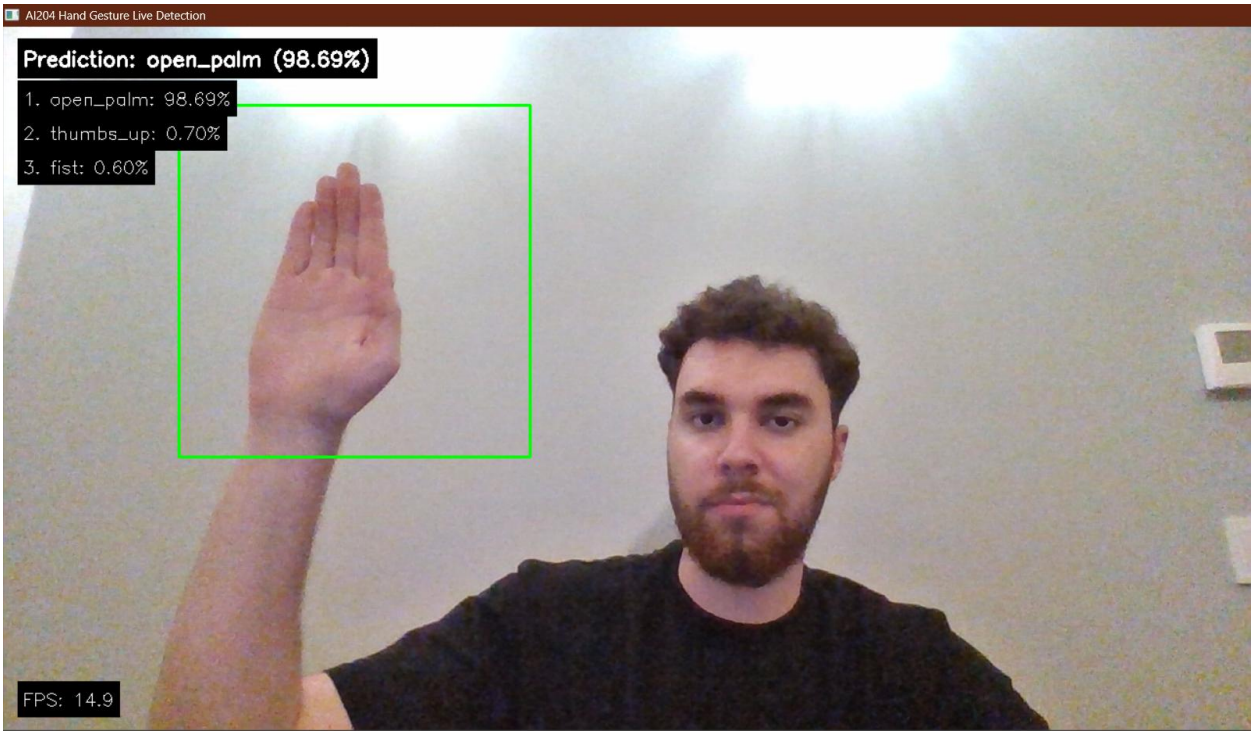
Prediction: thumbs_up (92.45%)

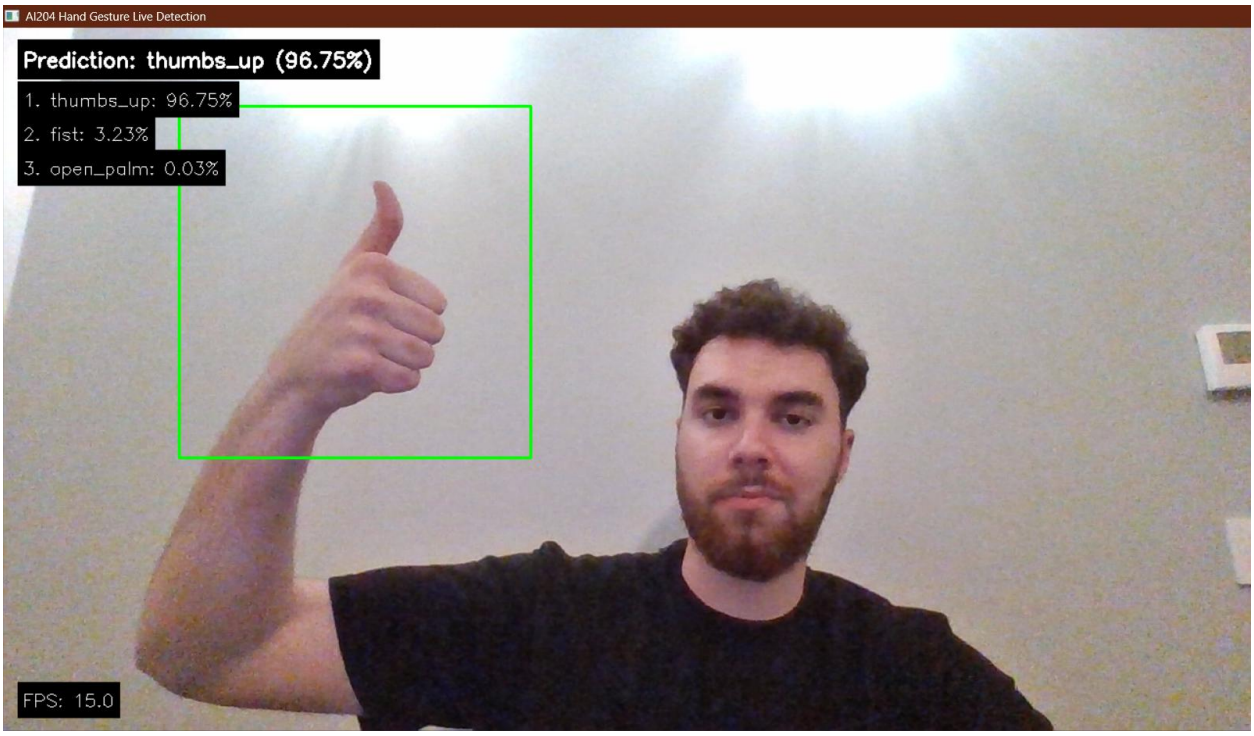
It also displays the top probabilities for the available classes, so the user can see not only the final prediction but also how confident the model is for each gesture.

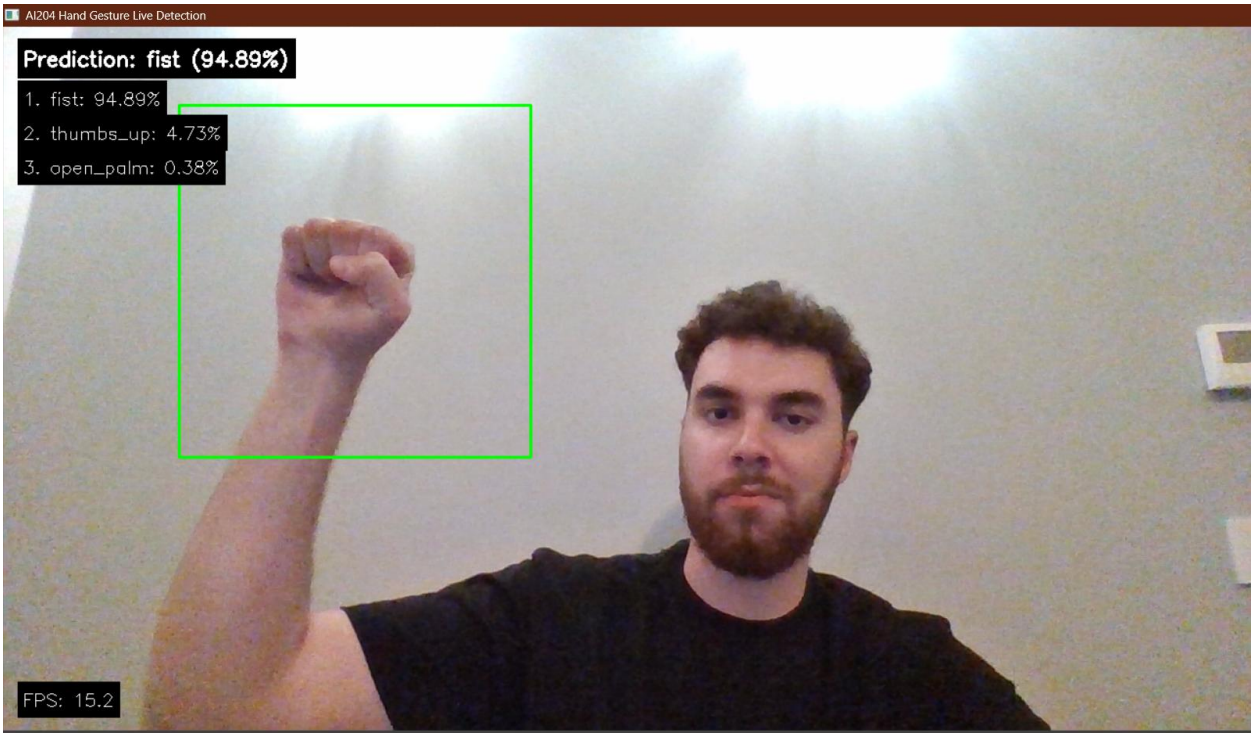
The script also includes practical controls. Pressing q closes the live detection window, while pressing s saves the current region of interest as a debug image. This is useful for checking whether the webcam crop and preprocessing are working correctly.

In my final implementation, the live detection script is mainly connected with the DCNN model, because the DCNN achieved much stronger test performance than the Vision Transformer. However, the script is general enough to load any compatible saved Keras model, as long as the model input size and class names match.

Some examples during live detection program run with DCNN:




AI204 Hand Gesture Live Detection
Prediction: thumbs_up (96.75%)
1. thumbs_up: 96.75%
2. fist: 3.23%
3. open_palm: 0.03%
FPS: 15.0


AI204 Hand Gesture Live Detection
Prediction: fist (94.89%)
1. fist: 94.89%
2. thumbs_up: 4.73%
3. open_palm: 0.38%
FPS: 15.2

In conclusion, this exercise implemented and compared two deep learning architectures for the same custom hand-gesture recognition task: a DCNN and a Vision Transformer. The DCNN achieved much stronger performance on the custom dataset and was therefore selected for the live webcam detection application. The Vision Transformer performed above random chance, but its lower accuracy suggests that it would require more data, pretraining, or architectural tuning to compete with the DCNN. Overall, the project covers both the convolutional network requirement and the Transformer requirement of the assignment, while also demonstrating a practical real-time detection system.